

ADVERSARIAL SEARCH

Game Playing • Minimax • Alpha-Beta Pruning • Evaluation Functions

Russell & Norvig | Two-player competitive games

How an AI chooses a move when an opponent is trying to defeat it

Learning Objectives

- Understand adversarial search and game-playing problems.
- Formulate a game using states, actions, players, terminal tests and utility.
- Explain the Minimax algorithm.
- Understand Alpha-Beta pruning and why it gives the same Minimax result.
- Understand evaluation functions for large game trees.
- Trace a simple game tree manually.

What Is Adversarial Search?

Search where another agent actively works against your objective.

NORMAL SEARCH

The environment does not deliberately oppose the agent.

ADVERSARIAL SEARCH

The opponent chooses actions that make your outcome worse.

Examples: Chess • Checkers • Tic-Tac-Toe • Connect Four

AI must ask: “What will my opponent do after my move?”

Game Formulation

Initial State

Starting board position

Players

MAX and MIN

Actions

Legal moves

Result

State after a move

Terminal Test

Is the game over?

Utility

Numeric outcome

A game can therefore be represented as a search tree.

MAX and MIN

MAX

Our AI player.

MAX tries to maximize the final utility.

MIN

Opponent.

MIN tries to minimize MAX's final utility.

MAX → best outcome for me

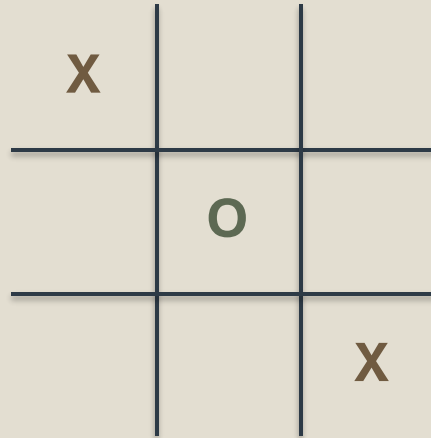
MIN → worst outcome for MAX

Minimax assumes the opponent plays rationally.

Tic-Tac-Toe as an Adversarial Search Problem

STATE

A 3×3 board configuration.



ACTIONS

Place your symbol in an empty square.

TERMINAL TEST → win / lose / draw

UTILITY → win = +1, draw = 0, loss = -1

Minimax — Core Idea

MAX NODE

Choose the child with the highest value.

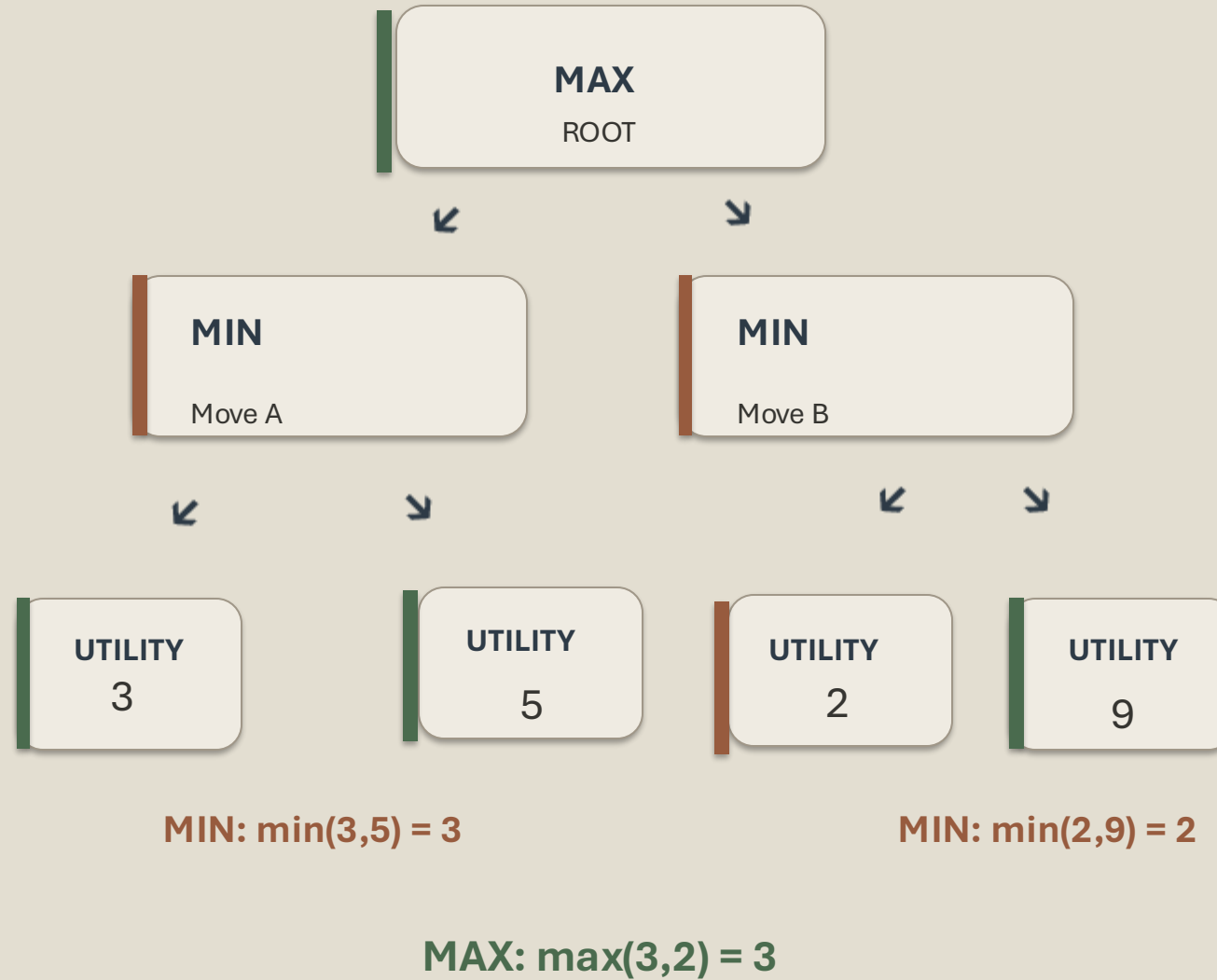
MIN NODE

Choose the child with the lowest value.

MAX = maximize | **MIN = minimize**

The value of a state depends on the opponent's best response.

Simple Minimax Game Tree



Minimax Algorithm — Step by Step

- Generate the game tree from the current state.
- Continue until terminal states or a chosen depth limit.
- Assign utility values to terminal states.
- At MIN nodes, select the minimum child value.
- At MAX nodes, select the maximum child value.
- Propagate the value back to the root and choose the corresponding move.

Minimax Recurrence

MAX

$\text{VALUE}(s) = \max \text{VALUE}(\text{successor})$

MIN

$\text{VALUE}(s) = \min \text{VALUE}(\text{successor})$

Terminal state → utility value

Minimax propagates utility values upward through alternating MAX and MIN levels.

Why Full Minimax Is Expensive

GAME TREE GROWTH

Every possible move creates more possible responses. Large games produce enormous trees.

PRACTICAL APPROACH

Search to a limited depth and use an evaluation function to estimate the position.

For branching factor b and search depth m :

Time $\approx O(b^m)$ | Space $\approx O(bm)$ with depth-first search

Alpha-Beta Pruning

Reduce unnecessary Minimax exploration.

KEY IDEA

If a branch cannot possibly affect the final decision, stop exploring it.

IMPORTANT

Alpha-Beta does not change the final Minimax answer. It only skips irrelevant branches.

SAME DECISION • FEWER NODES

α = best value MAX can guarantee so far

β = best value MIN can guarantee so far

Alpha-Beta: When Do We Prune?

ALPHA (α)

Best value MAX can guarantee so far along the current path.

BETA (β)

Best value MIN can guarantee so far along the current path.

PRUNE when $\alpha \geq \beta$

Further exploration cannot improve the decision at the relevant ancestor.

Alpha-Beta Example

MAX
ROOT

MIN
LEFT

MIN
RIGHT

VALUE
3

VALUE
5

VALUE
2

VALUE
?

LEFT MIN = $\min(3,5) = 3$

MAX now has $\alpha = 3$

**RIGHT MIN sees 2 $\rightarrow \beta = 2$
 $\alpha \geq \beta \rightarrow$ prune remaining branch**

Evaluation Functions

WHY?

The complete game tree may be too large. Stop at a cutoff depth and estimate the position.

WHAT?

A numeric estimate of how good a non-terminal state is for MAX.

Higher score → better for MAX | Lower score → better for MIN

A good evaluation function should be fast to compute and useful for distinguishing positions.

Example Evaluation Function — Chess

MATERIAL SCORE

Queen = +9

Rook = +5

Bishop/Knight = +3

Pawn = +1

OTHER FEATURES

Mobility • King safety • Pawn structure • Position •
Threats

Evaluation = weighted combination of useful features.

The exact evaluation function depends on the game and the designer.

Minimax vs Alpha-Beta vs Evaluation

MINIMAX

Searches the game tree.

MAX → maximize

MIN → minimize

ALPHA-BETA

Optimizes Minimax by pruning branches that cannot affect the result.

EVALUATION

Estimates non-terminal positions when full search is impossible.

Together: efficient game-playing systems can search selectively and evaluate positions.

Exam / Viva Focus

- What is adversarial search?
- How is a game formulated as a search problem?
- What are MAX and MIN?
- Explain Minimax with a game tree.
- Why is full Minimax expensive?
- What is Alpha-Beta pruning?
- Define α and β . When can pruning occur?
- Does Alpha-Beta change the Minimax result?
- What is an evaluation function and why is it needed?

FINAL TAKEAWAY

ADVERSARIAL SEARCH = search with an opponent

GAME → states + actions + terminal test + utility

MINIMAX → MAX maximizes, MIN minimizes

ALPHA-BETA → same answer, fewer branches evaluated

EVALUATION → estimates non-terminal positions

Think: “My move is good only if it remains good after the opponent's best response.”